ESIEE Paris

E3 - FI

Rapport de Travaux Pratiques 1

Traitement du Signal

Auteur : Félix MIELCAREK

Destinataire : Hassane MIMOUN - Philippe TOURON

Date : 9 mars 2026

Table des matières

1 Introduction **2**
1.1 Contexte et environnement du TP 2
1.2 Raison d'être du rapport 2

2 Méthodologie de travail **2**

3 Travail réalisé **2**
3.1 Exercice 1 2
3.1.1 Code 2
3.1.2 Explications 3
3.2 Exercice 2 3
3.2.1 Code 3
3.2.2 Explications 4
3.3 Exercice 3 4
3.3.1 Code 4
3.3.2 Explications 5

4 Résultats obtenus **6**
4.1 Résultats de l'exercice 1 6
4.1.1 Partie 1 : Fonction sinc() de MATLAB 6
4.1.2 Partie 2 : Calcul manuel avec instruction IF 7
4.2 Résultats de l'exercice 2 7
4.3 Résultats de l'exercice 3 8

5 Acquis durant le TP **8**
5.1 Connaissances techniques 8
5.2 Difficultés rencontrées et solutions 8
5.3 Apprentissages et perspectives 9

1 Introduction

1.1 Contexte et environnement du TP

Ce travail pratique constitue une première prise en main de MATLAB, à la fois en tant que logiciel de calcul numérique et en tant que langage de programmation. MATLAB est un environnement largement utilisé dans le domaine du traitement du signal et de l'analyse scientifique.

1.2 Raison d'être du rapport

Ce rapport vise à rendre compte de la découverte et de l'apprentissage de MATLAB à travers la réalisation d'exercices pratiques. Il présente les différentes étapes de travail ainsi que les résultats obtenus.

2 Méthodologie de travail

La méthodologie adoptée pour ce TP s'articule autour de plusieurs étapes. Dans un premier temps, une lecture attentive du manuel MATLAB fourni a permis de se familiariser avec les concepts fondamentaux et la syntaxe du langage. Ensuite, les exercices ont été réalisés et testés les uns après les autres, chacun permettant d'approfondir des aspects spécifiques de MATLAB. En cas de besoin, la documentation officielle en ligne a été consultée pour clarifier certains détails de fonctionnement ou découvrir des fonctionnalités supplémentaires.

3 Travail réalisé

3.1 Exercice 1

3.1.1 Code

```
clear; clc;

figure;
x = -1:0.1:1;
y = sinc(x);
plot(x, y, 'b-', 'LineWidth', 1.5);
grid on;
axis([-1, 1, -0.5, 1.2]);
xlabel('x');
ylabel('sinc(x)');
title('Fonction Sinus cardinal - Partie 1');

figure;
x = linspace(-1, 1, 100);
y = zeros(1, length(x));
for i = 1:length(x)
if x(i) == 0
```

```
y(i) = 1;
else
y(i) = sin(pi _ x(i)) / (pi _ x(i));
end
end
plot(x, y, 'r-', 'LineWidth', 1.5);
grid on;
axis([-1, 1, -0.5, 1.2]);
xlabel('x');
ylabel('sin(pi x)/(pi x)');
title('Fonction Sinus cardinal - Partie 2 (avec IF)');
```

Listing 1 – Code de l'exercice 1

3.1.2 Explications

Logique du code :

L'exercice compare deux méthodes pour tracer la fonction sinus cardinal. La partie 1 utilise directement la fonction `sinc()` intégrée à MATLAB (Signal Processing Toolbox), qui calcule automatiquement $\frac{\sin(\pi x)}{\pi x}$ avec gestion du cas $x = 0$. La partie 2 implémente manuellement le même calcul en utilisant une boucle `for` et l'instruction `IF` pour éviter la division par zéro.

Algorithme :

Partie 1 : Génération d'un vecteur x sur l'intervalle $[-1, 1]$, appel direct de la fonction `sinc()`, tracé du graphique avec grille.

Partie 2 : Génération d'un vecteur x avec `linspace` (100 points), initialisation du vecteur y, boucle `for` itérant sur chaque élément, test `IF` pour détecter $x = 0$ (affectation de 1) ou calcul de $\sin(\pi x)/(\pi x)$ sinon.

Choix de conception :

- Deux figures distinctes pour comparer visuellement les résultats des deux méthodes.
- Partie 1 : Utilisation de la fonction MATLAB native comme demandé dans l'énoncé.
- Partie 2 : Instruction `IF` explicite dans la boucle pour la gestion du cas limite.
- Axes identiques $[-1, 1] \times [-0.5, 1.2]$ pour faciliter la comparaison.
- Grilles activées sur les deux graphiques pour une meilleure lecture.
- Couleurs différentes (bleu/rouge) pour distinguer les deux approches.

3.2 Exercice 2

3.2.1 Code

```
% Code de l'exercice 2
clear; clc;

%% Generation des matrices magiques
for a = 5:2:11
eval(sprintf('M%d = magic(%d);', a, a));
fprintf('Matrice magique M%d creee\n', a);
end

%% Affichage des matrices
```

```
fprintf('\n--- Affichage des matrices magiques ---\n\n');

fprintf('Matrice M5 (5x5) :\n');
disp(M5);

fprintf('Matrice M7 (7x7) :\n');
disp(M7);

fprintf('Matrice M9 (9x9) :\n');
disp(M9);

fprintf('Matrice M11 (11x11) :\n');
disp(M11);
```

Listing 2 – Code de l'exercice 2

3.2.2 Explications

Logique du code :

Le programme génère quatre matrices magiques de tailles impaires (5x5, 7x7, 9x9, 11x11). Une matrice magique est un carré où la somme de chaque ligne, colonne et diagonale est identique.

Algorithme :

Boucle `for` avec pas de 2 pour générer uniquement les dimensions impaires de 5 à 11. Utilisation de `eval()` et `sprintf()` pour créer dynamiquement des variables M5, M7, M9, M11. Chaque matrice est générée via la fonction `magic()` de MATLAB.

Choix de conception :

- Utilisation de `eval()` pour créer des variables avec des noms dynamiques.
- Boucle avec pas de 2 (5 :2 :11) pour ne générer que les tailles impaires demandées.
- Affichage formaté avec `fprintf()` et `disp()` pour une sortie claire et structurée.

3.3 Exercice 3

3.3.1 Code

```
% Code de l'exercice 3
clear; clc;

%% Parametres
epsilon = 1e-4;
n = 1;
somme = 0;
historique_somme = [];

%% Calcul de la somme
while (1/n^2) > epsilon
somme = somme + (1/n^2);
historique_somme(n) = somme;
n = n + 1;
end
```

```

%% Affichage des resultats
N = n;
fprintf('Le plus petit entier N tel que 1/N^2 <= %f est : %d\n', epsilon, N);
fprintf('La somme totale est : %f\n', somme);

%% Visualisation graphique
figure;
plot(1:length(historique_somme), historique_somme, 'b-', 'LineWidth', 1.5);
grid on;
xlabel('Valeur de n');
ylabel('Somme cumulee');
title(['Evolution de la somme pour epsilon = ', num2str(epsilon)]);
```

Listing 3 – Code de l'exercice 3

3.3.2 Explications

Logique du code :

Le programme calcule la somme des termes $1/n^2$ jusqu'à ce que le terme suivant soit inférieur à ε. Cette somme converge vers $\pi^2/6$ d'après le résultat de Leonhard Euler.

Algorithme :

Boucle `while` qui s'arrête lorsque $1/n^2 \leq \varepsilon$. À chaque itération : ajout du terme $1/n^2$ à la somme, sauvegarde dans l'historique, incrémentation de n. L'historique permet de tracer l'évolution de la convergence.

Choix de conception :

- Utilisation d'un vecteur *historique_somme* pour conserver toutes les valeurs intermédiaires.
- ε défini comme paramètre pour faciliter les tests avec différentes précisions.
- Graphique pour visualiser la convergence de la série.
- Sections commentées (%%) pour structurer le code de manière claire.

4 Résultats obtenus

4.1 Résultats de l'exercice 1

4.1.1 Partie 1 : Fonction sinc() de MATLAB

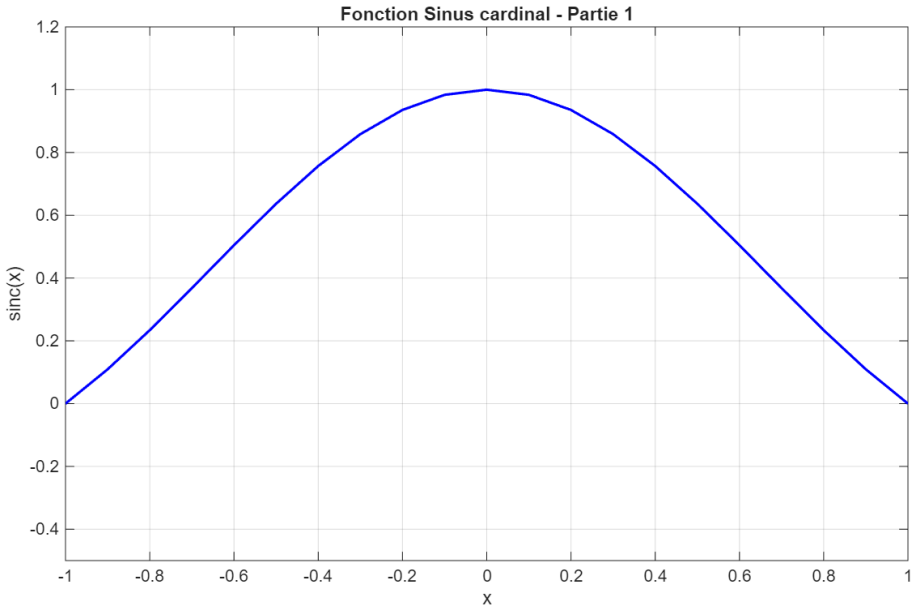

FIGURE 1 – Tracé de la fonction sinus cardinal avec la fonction intégrée `sinc()`

Le graphique montre la fonction sinus cardinal calculée avec la fonction `sinc()` de MATLAB. On observe la forme caractéristique de cette fonction avec un maximum de 1 en $x = 0$ et des oscillations amorties qui décroissent de part et d'autre.

4.1.2 Partie 2 : Calcul manuel avec instruction IF

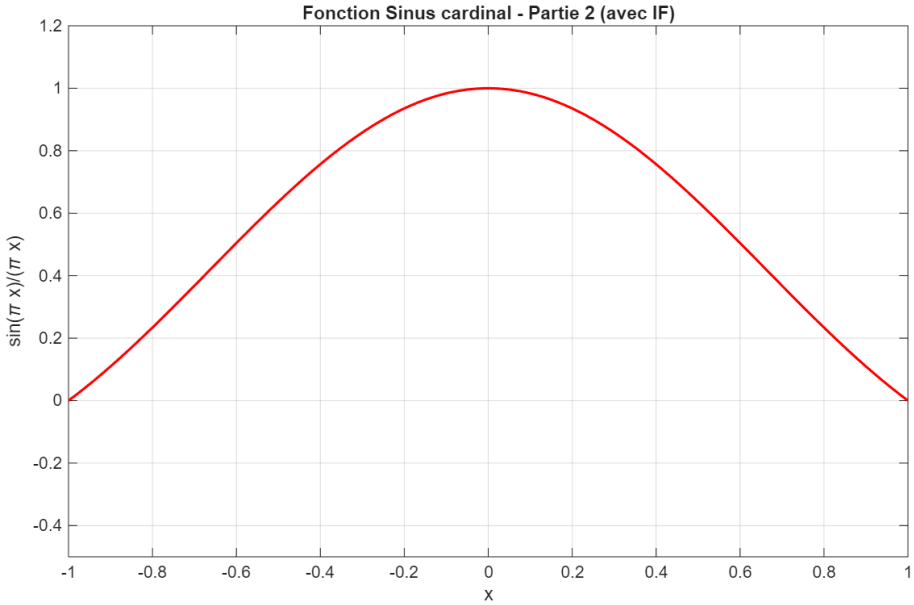

FIGURE 2 – Tracé de la fonction sinus cardinal avec calcul manuel et instruction `IF`

Ce graphique présente le même résultat obtenu par calcul manuel. La superposition mentale des deux courbes confirme que l'implémentation avec la boucle `for` et l'instruction `IF` reproduit fidèlement le comportement de la fonction `sinc()`. La gestion du cas $x = 0$ par l'instruction `IF` permet d'éviter la division par zéro.

4.2 Résultats de l'exercice 2

L'exécution du programme a généré les quatre matrices magiques demandées. Voici un extrait de la sortie du terminal :

```
Matrice magique M5 créée
Matrice magique M7 créée
Matrice magique M9 créée
Matrice magique M11 créée

--- Affichage des matrices magiques ---

Matrice M5 (5x5) :
17 24 1 8 15
23 5 7 14 16
4 6 13 20 22
10 12 19 21 3
11 18 25 2 9
```

Les quatre matrices magiques (M5, M7, M9, M11) ont été créées avec succès. La matrice M5 présentée ci-dessus illustre la propriété fondamentale des carrés magiques : la somme de chaque ligne, colonne et diagonale principale est égale à 65. Cette valeur, appelée constante magique, est calculée par la formule $\frac{n(n^2+1)}{2}$ où n est l'ordre de la matrice.

L'utilisation combinée de la boucle `for` avec pas de 2, de `eval()` pour la génération dynamique de variables, et de `magic()` démontre la puissance de MATLAB pour la manipulation matricielle et la programmation dynamique.

4.3 Résultats de l'exercice 3

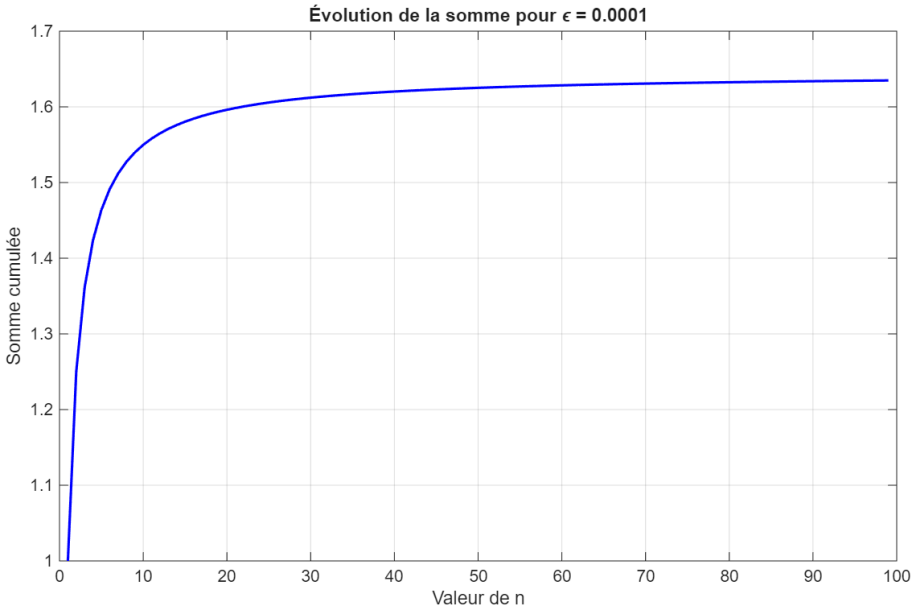

FIGURE 3 – Évolution de la somme cumulée $\sum *k = 1^n \frac{1}{k^2}$ en fonction de n

Le graphique illustre la convergence de la série vers une valeur limite (environ 1.64, correspondant à $\pi^2/6 \approx 1.6449$). La boucle `while` s'arrête lorsque le terme $1/n^2$ devient inférieur à $\varepsilon = 10^{-4}$. On observe une croissance rapide au début puis un ralentissement progressif, caractéristique d'une convergence asymptotique.

5 Acquis durant le TP

5.1 Connaissances techniques

Ce travail pratique a permis une première prise en main approfondie de MATLAB. Les principales compétences acquises incluent :

- La syntaxe de base du langage MATLAB (déclaration de variables, boucles, conditions)
- L'utilisation des structures de contrôle (`if`, `for`, `while`)
- La manipulation de vecteurs et matrices
- La création et personnalisation de graphiques (`plot`, `grid`, `xlabel`, etc.)
- L'utilisation de fonctions intégrées (`sinc`, `magic`, `linspace`)
- La génération dynamique de variables avec `eval()` et `sprintf()`

5.2 Difficultés rencontrées et solutions

La principale difficulté rencontrée concerne les spécificités du langage MATLAB, notamment l'affectation et la manipulation de fonctions dans des variables. Contrairement

à d'autres langages, MATLAB possède des particularités dans la gestion des handles de fonctions et l'évaluation dynamique de code. Ces aspects ont nécessité une consultation régulière de la documentation pour bien comprendre le fonctionnement des commandes comme `eval()`. La pratique progressive à travers les trois exercices a permis de surmonter ces difficultés.

5.3 Apprentissages et perspectives

Ce TP a révélé que MATLAB est un outil particulièrement adapté au traitement du signal. Sa syntaxe orientée matrices, ses fonctions mathématiques intégrées et ses capacités de visualisation en font un environnement idéal pour ce domaine. Les notions découvertes durant ce TP constituent une base solide pour aborder des problématiques plus avancées en traitement du signal, analyse de données et calcul scientifique. L'apprentissage de MATLAB sera un atout précieux pour les projets futurs dans le domaine de l'ingénierie du signal.